

LLD – Engagement Module (OpenScholar)

1. Overview

This document defines the Low-Level Design (LLD) for the Engagement module of OpenScholar. The module is responsible for handling user interactions with papers, including reactions and comments with one-level reply support. The design ensures performance, data integrity, and scalability for high-read and write workloads.

2. Scope

The Engagement module is responsible for:

- Add reaction to a paper (one per user per paper)
- Remove or update reaction
- Add comment to a paper
- Reply to a comment (one-level nesting)
- Retrieve comments for a paper
- Maintain engagement counts (optional aggregation)

Out of Scope:

- Notifications (handled in Notification module)
 - Moderation of comments (handled in Moderation module)
-

3. Data Model (Relevant Tables)

Tables used:

- reactions
- comments
- users
- papers

reactions

- user_id (UUID)
- paper_id (UUID)
- type (VARCHAR)
- created_at (TIMESTAMP)

Constraints:

- PRIMARY KEY (user_id, paper_id)

comments

- id (UUID)
- paper_id (UUID)
- user_id (UUID, nullable for guest)
- content (TEXT)
- parent_id (UUID, nullable)
- is_deleted (BOOLEAN)
- created_at (TIMESTAMP)

Constraints:

- parent_id references comments.id
- Only one-level nesting allowed

4. API Contracts

4.1 Add or Update Reaction

Endpoint: POST /api/reactions

Request Body: { "paperId": "uuid", "type": "like" }

Response: { "message": "Reaction recorded" }

Behavior:

- If reaction exists → update type
- Else → insert new reaction

4.2 Remove Reaction

Endpoint: DELETE /api/reactions

Request Body: { "paperId": "uuid" }

Response: { "message": "Reaction removed" }

4.3 Add Comment

Endpoint: POST /api/comments

Request Body: { "paperId": "uuid", "content": "This is a comment" }

Response: { "commentId": "uuid" }

4.4 Reply to Comment

Endpoint: POST /api/comments/reply

Request Body: { "paperId": "uuid", "parentId": "comment_uuid", "content": "Reply text" }

Response: { "commentId": "uuid" }

4.5 Get Comments for Paper

Endpoint: GET /api/comments?paperId=<uuid>

Response: { "comments": [{ "id": "uuid", "content": "text", "user": { "id": "uuid", "name": "User" }, "replies": [{ "id": "uuid", "content": "reply" }] }] }

5. Service Logic

5.1 addOrUpdateReaction()

Steps: 1. Validate user authentication 2. Check if reaction exists for (user_id, paper_id) 3. If exists → update type 4. Else → insert new reaction

5.2 removeReaction()

Steps: 1. Validate user 2. Delete reaction by (user_id, paper_id)

5.3 addComment()

Steps: 1. Validate input 2. Insert new comment (parent_id = null)

5.4 replyToComment()

Steps: 1. Validate parent comment exists 2. Ensure parent has no parent (one-level restriction) 3. Insert reply with parent_id

5.5 getComments()

Steps: 1. Fetch top-level comments (parent_id = null) 2. Fetch replies for each comment (batch query) 3. Join user info 4. Return structured response

6. Validation Rules

- reaction type must be predefined enum (like, love, etc.)
 - content must not be empty
 - parent comment must exist
 - parent comment must not already be a reply
-

7. Performance Considerations

- Use index on paper_id for comments
 - Use index on parent_id for replies
 - Use index on (user_id, paper_id) for reactions
 - Batch fetch replies to avoid N+1 queries
-

8. Security Considerations

- Only authenticated users can react
 - Comments can allow guest users (optional)
 - Validate ownership before delete/update
-

9. Edge Cases

- Multiple reactions from same user (prevented by PK)
 - Reply to reply (should be rejected)
 - Deleted comment with replies
 - Empty content submission
-

10. Prisma Model Mapping

```
model Reaction { userId String paperId String type String createdAt DateTime @default(now())
```

```
@@id([userId, paperId]) }
```

```
model Comment { id String @id @default(uuid()) paperId String userId String? content String parentId String? isDeleted Boolean @default(false) createdAt DateTime @default(now()) }
```

11. Folder Structure

```
src/ app/api/comments/route.ts app/api/comments/reply/route.ts app/api/reactions/route.ts
```

```
modules/engagement/service.ts modules/engagement/repository.ts
```

12. Implementation Notes

- Use soft delete for comments
 - Avoid deep nesting to simplify queries
 - Use Prisma transactions if needed for batch operations
-

13. Acceptance Criteria

- User can react to paper (only once)
 - User can update/remove reaction
 - User can add comment
 - User can reply (one level)
 - Comments load efficiently with replies
-

14. Future Enhancements

- Reaction counts caching
 - Comment likes
 - Nested comments (multi-level)
 - Real-time updates (WebSocket)
-

End of Document